

Figure 4: CoLT for set-associative L1 and L2 TLBs.

Figure 5: CoLT for fully-associative TLBs.

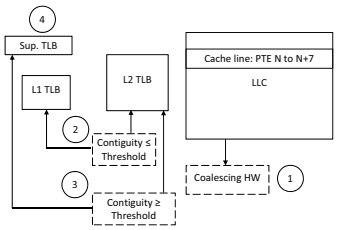


Figure 6: Combined CoLT for all TLBs.

lookup latencies. Third, coalescing is *unintrusive*, unlike speculation and prefetching [6, 11, 19, 27] which can degrade performance. For example, incorrect speculations suffer a high penalty. Incorrect prefetches lead to the eviction of useful entries and higher bandwidth usage. Prior work mitigates these problems by using separate structures to store prefetched translations or perform speculation [6, 11, 19]. In contrast, we coalesce entries directly into the TLBs but ensure that coalescing occurs only around *on-demand* translations. In the worst case, coalesced entries may be unused but are not harmful. This is crucial given that system contiguity does not necessarily imply that all contiguous translations are used in temporal proximity. We ensure coalesced entries are available if needed but do not harm TLB hit rates when they are unused.

We propose three variants of CoLT for two-level TLB hierarchies. This hierarchy contains set-associative L1 TLB and L2 TLBs, used to cache baseline 4KB pages [3, 17]. Superpages are cached in separate small, fully-associative TLBs that are accessed in parallel with the L1 TLB. Note that the L2 TLB is inclusive of just the set-associative L1 TLB and not the superpage TLB.

There are three natural coalescing mechanisms for this hierarchy. First, we coalesce in just the set-associative L1 and L2 TLBs. Second, we coalesce in the superpage TLB only. Third, we use a combined approach that routes some coalesced entries to the set-associative TLBs and others to the superpage TLBs. We now describe each of these schemes.

4.1. CoLT-SA Design and Implementation

CoLT-Set Associative (CoLT-SA) coalesces multiple virtual-to-physical page translations in the set-associative L1 and L2 TLBs. We first detail its high-level operation and then focus on specific design challenges.

4.1.1. Overall operation. The bottom half of Figure 4 shows a high-level view of CoLT-SA. In step 1, the set-associative L1 TLB and superpage TLB are looked up in parallel. Assuming L1 and L2 TLB misses (step 2), a page table walk brings in the desired translation entry into the LLC (step 3). We assume, like most x86 systems with dedicated MMU page table caches [5], that the LLC is the highest cache level for page table entries.

After the LLC fill, two parallel events occur. First, the requested translation is returned to the processor pipeline. In parallel, the *Coalescing Logic* studies the translations around the requested entry for contiguity. It coalesces as many of these translations as possible, as long as they map to the same set. This coalesced entry is inserted into the L1 and L2 TLBs (step 4). However, conventional set-associative TLBs map consecutive virtual addresses (and hence contiguous translations) to consecutive sets, precluding coalescing. We therefore modify the virtual page bits used for set-selection so that translations for groups of consecutive virtual page numbers do map to the same set. Furthermore, since we provide the requested translation to the pipeline in parallel with the *Coalescing Logic's* operation, the latter is off the critical path and does not affect TLB miss handling times.

4.1.2. TLB set selection. To understand how we modify TLB set selection to permit coalescing, consider the following example. An 8-set TLB would require three bits, bits 2 to 0 of the virtual page number for set selection (VPN[2-0]). Naturally, this would map consecutive translations to consecutive sets, preventing coalescing. However, if we left-shift the index bits by $log_2(N)$ bits, we may place N consecutive translations in the same set (permitting a maximum of N contiguous translations to be coalesced into a single entry). Therefore, to ensure that translations with four consecutive virtual pages map to the same set, we use VPN[4-2] as the new indexing bits.

To coalesce more entries, the indexing bits are further left-shifted (for example, to coalesce up to eight entries, VPN[5-3] must be used). However, using higher order bits for set indexing increases conflict misses since more consecutive entries are mapped to the same set. This is a fundamental tradeoff for CoLT-SA designs – in choosing the correct index bits, we must balance opportunities for coalescing with potentially higher conflict misses. We will show that allowing for coalescing of four contiguous translations generally performs best.

4.1.3. Lookup operation. The top half of Figure 4 illustrates CoLT-SA lookups. Each coalesced TLB entry maintains tag bits, the higher order bits left of the index bits used for set selection. For example, if up to four contiguous translations can be coalesced in a TLB with eight sets, VPN[4-2] is used for set selection and VPN[63-5] is the tag. In step (a), this tag is checked against the requested virtual page number. In step (b), the non-index lower-order virtual page bits (VPN[1-0] in our example) are used to select among multiple valid bits. There is one valid bit for every possible translation in a coalesced entry. These valid bits indicate the presence of a translation in the coalesced entry. If on step (b), a valid bit is set, there is a TLB hit. At this point, extra logic calculates the physical page number. CoLT entries store the base physical page number for each coalesced entry. This number corresponds to the virtual page represented by the first set valid bit. To reconstruct the physical page number, combinational logic (*PPN Generation Logic*) calculates the number of valid bits away this entry is from the first set valid bit. This number is added to the stored base physical page number to yield the desired physical page.

4.1.4. Practical coalescing restrictions. Ideally, after the page table is walked to handle a TLB miss, coalescing logic finds as many contiguous translations around the requested translation as possible. Practically, however, coalescing is restricted by two constraints. First, as we have already discussed, the choice of index bits for set selection places a limit on coalescing opportunity. A second limit arises from our desire to minimize the overhead associated with